27 · 资源审查与 TiDB Operator 迁移（方案 C）实录

项	内容
文档编号	DT-PdM-27
日期	2026-10-22
结论	迁移未完成，已安全回滚；数据零丢失（1 亿行核对一致）；服务已恢复
验证	E2E 66/66 通过（回滚后）

---

1. 全量资源审查结果

项	数据
命名空间	38 个
Pod 总数	236（Running 180 / Completed 43 / Error 10 / Pending 2 / CrashLoop 1）
节点	仅 2 个：k8s-master 16C / k8s-worker1 64C
CPU requests	master 94%（15.1C/16C） · worker1 97%（62.4C/64C）
PV	889 个（886 Bound / 2 Available / 1 Released）

CPU 占用 Top 6：dify 6.15C · infra 5.71C · qianrenqianmian 4.75C · dataagent-course 2.5C · pdm-prod 1.5C · kube-system 1.35C

1.1 清理清单（仅删明确损坏/遗留，不碰健康业务）

类别	对象	释放
失败 Job Pod	dify `obsidian-dify-sync-*` ×10（Error）	0（已终止）
遗留 Pod	全局 43 个 Completed	0（已终止）
损坏 StatefulSet	`monitoring/loki-stack`（重启 119 次，已 Completed）	0
失败迁移	`tidb-new/pdm-new` TidbCluster（PD 0/3）	~1.6C
Pending Pod	`kube-system/docker-registry-master`	0

关键发现：所有"异常"Pod 都是已终止状态（Completed/Error），不占 CPU。清理只能净化，无法腾出容量。

真正的容量来源是：停掉旧 TiDB 集群（5.6C），这也正是方案 C 的前提。

---

2. 方案 C 可行性验证（先讲结论）

「原地接管 PV」对 PD 不可行，理由：

- PD 成员身份绑定在 `name + DNS`（etcd peer 身份），写入数据目录
- Operator 生成的名称必然不同：`pdm-pd-0` vs 旧 `pd-0`，Service `pdm-pd-peer` vs `pd-peer`

- 换名后 PD 无法启动（etcd 拒绝不同 member 身份复用同一数据目录）
- 而新建 PD 也读不懂旧 TiKV 数据——region 元数据存在 PD 里，不在 TiKV

所以方案 C 的可行形式是：原地停机 → 重建受管集群 → BR 恢复。效果等同，且无需额外容量（旧集群停掉即释放）。

---

3. 执行过程与四个阻塞

3.1 已完成部分

步骤	结果
迁移前全量备份	4.29 GB / 60.5s → `s3://pdm-backup/pre-migration-20261022`（199 objects，含 backupmeta，已校验）
数据基线记录	21 张表，feature_point=100,000,000
停应用 + 停旧集群	释放 8.25C（worker1 62.4C→55.4C）
删除旧 STS/PVC	7 个 PV 转 Released → 清 claimRef → Available
TiDB Operator 安装	v1.6.6，controller-manager 1/1 Running
TidbCluster 创建	PD 3/3 起来过，PVC 全部绑定到原 PV

3.2 四个阻塞（解决 3 个）

#	现象	根因	状态
1	`Fatal glibc error: CPU does not support x86-64-v2`	新镜像要求 SSE4.2，Xeon 5320 不支持	改用 `-x86v1` 变体
2	`set: Illegal option -o pipefail`	镜像 `/bin/sh`→dash，operator 脚本用 bash 语法	构建 `ln -sf /bin/bash /bin/sh`
3	`nslookup ... failed` 循环	x86v1 是 Ubuntu 22.04，无 `nslookup`；且 operator 把启动脚本挂到 `/usr/local/bin`，遮住放在那里的 shim	shim 改放 `/usr/bin`
4	PD 永久卡在 `waiting for discovery service to return start args`	discovery 服务 ↔ PD 互等死锁	未解决

3.3 阻塞 4 的精确证据

```
PD:        waiting for discovery service to return start args ...
discovery: failed to discover: pdm-pd-1.pdm-pd-peer.infra.svc.cluster.local,
           Get "http://pdm-pd.infra:2379/pd/api/v1/members": connect: connection refused
```

- PD 等 discovery 给启动参数
- discovery 反过来要查 PD 的 `/pd/api/v1/members` 才能决定「bootstrap 还是 join」
- PD 没起来 → 查不到 → discovery 报错而不是返回 bootstrap 参数 → PD 继续等

- 30 秒超时后 PD 退出重启，无限循环

反证：同一套 PV、同样的镜像，改用手管 StatefulSet（直接传 `--initial-cluster=pd-0=...,pd-1=...,pd-2=...`，绕过 discovery）后，PD 3/3 秒起。说明数据和镜像都没问题，问题在 operator 的 discovery 引导链路。

尝试过的修复（均无效）：

- 清空 PD 数据目录
- TC 缩容到 1 个 PD 单独 bootstrap
- 清空 TC `status.pd.members`
- 重启 discovery Pod
- 检查 RBAC（Role 有 `tidbclusters get`，正常）

---

4. 回滚与数据恢复

4.1 回滚动作

- 删除 TidbCluster `pdm`（Operator 生成的 STS/PVC 一并清除）
- 清 PV `claimRef` → 7 个 PV 全部 Available
- `kubectl apply -f tidb-cluster.yaml`（原手管清单）
- PD/TiKV/TiDB 全部 Running

4.2 BR 恢复

宿主机跑 br 解析不了集群 DNS，改为把 br 二进制拷进 PD Pod 内执行：

```
br restore db --db pdm --pd pd-0.pd-peer.infra.svc.cluster.local:2379 \
  --storage "s3://pdm-backup/pre-migration-20261022?endpoint=http://milvus-minio.infra.svc.cluster.local:9000&..."
```

指标	值
结果	Restore success，195 ranges succeed / 0 failed
数据量	100,059,802 KV / 30.04 GB（压缩后 4.29 GB）
耗时	1m27s，平均 343.9 MB/s

4.3 数据核对（21 张表逐一比对）

```
feature_point              100,000,000 → 100,000,000  OK
asset                        2,400 → 2,400  OK
measurement_point           20,400 → 20,400  OK
asset_metric_snapshot        2,400 → 2,400  OK
diagnosis                      528 → 528  OK
recommendation_card             20 → 20  OK
read_asset_health_history       43 → 43  OK
processed_event                331 → 331  OK
...■■ 21 ■■■■ OK■
```

4.4 恢复后发现的坑

`pdm_diag` 用户丢失 —— 新集群是全新 bootstrap，`mysql` 系统库不在 BR 恢复范围（只恢复了 `pdm` 库），所以数据库用户没了。应用报 `Access denied`。

修复：重建用户 + 授权。

```
CREATE USER IF NOT EXISTS 'pdm_diag'@'%' IDENTIFIED BY '<secret>';
GRANT SELECT,INSERT,UPDATE,DELETE,CREATE,DROP,INDEX,ALTER ON pdm.* TO 'pdm_diag'@'%';
```

> 这条要写进迁移 runbook：BR 只恢复业务库，账号/授权/全局变量需要单独迁移。

5. 最终状态

```
■■    pd 3/3 · tikv 3/3 · tidb-server 2/2 · tiflash 1/1  Running
■■    1 ■■■■■■21 ■■■■■■■
■■    python-diagnosis×2 · java-asset · pulsar-consumer · go-eventpipe
        integration-worker · mock-eam  ── ■■ Running
■■    E2E 66/66 ■■
■■    worker1 CPU requests 60.4C/64C (94%)■■■■■■■■■
```

TiDB Operator v1.6.6 保留安装（`tidb-admin`），可用于后续备份/监控编排，不影响现有集群。

6. 结论与建议

方案 C 在当前环境不可行，卡在 TiDB Operator 的 discovery 引导链路。这不是配置问题，是 operator 与这套 x86v1 镜像/网络环境的兼容性问题。

三条后续路径：

路径	说明	风险
A. 查清 discovery 死锁	需读 operator 源码定位 `server.go:115` 的分支逻辑，可能要打 patch	时间不可控
B. 保持手管 + 用 Operator 只做备份/监控	已具备（Operator 已装，可加 `BackupSchedule`）	低，推荐
C. 升级到 Operator v2.x	需先升 K8s 到 ≥1.30	大版本升级，风险高

我的建议是 B：手管集群已验证稳定（1 亿行、E2E 66/66），而 Operator 真正带来的增量价值是自动化备份和监控——这部分不需要它接管 PD/TiKV 生命周期。

7. 本次操作的经验（写给下一次）

- 停集群顺序：先停 TiKV，等 region 交接完，再停 PD。我这次先停 PD，TiKV 卡在 Terminating，最后靠 `--force` 强杀。
- BR 只恢复业务库，用户/授权/全局变量要单独处理。
- br 要在集群内跑，宿主机跑解析不了 `*.svc.cluster.local`。
- operator 把启动脚本挂到 `/usr/local/bin`，往镜像里放工具别放这个目录。
- x86v1 镜像是 Ubuntu，与官方 Alpine 镜像差异较大（无 nslookup、/bin/sh 是 dash）。
- 变更前一定先记录逐表行数基线，这次靠它 5 分钟就确认了数据无损。